

Laboratorio 8 - Máquinas Vectoriales de Soporte (SVM)

Bryan Martínez, Adriana Palacios, y Nery Molina

2026-04-26

Repositorio

https://github.com/Bamo0507/Laboratorio8_SVM

Actividad 1 — Conjuntos de Entrenamiento y Prueba

Se utilizan los mismos conjuntos de entrenamiento y prueba que han sido empleados en los laboratorios anteriores. Se mantienen en una partición 70-30 sobre el dataset de propiedades de Airbnb. Esto nos permite que los modelos generados sean comparables, al garantizar que se están trabajando bajo las mismas condiciones.

```
cat("Total de filas en el dataset:", nrow(listings_model), "\n")
```

```
## Total de filas en el dataset: 76246
```

```
cat("Datos de entrenamiento:", nrow(train_num), "\n")
```

```
## Datos de entrenamiento: 43729
```

```
cat("Datos de prueba:", nrow(test_num), "\n")
```

```
## Datos de prueba: 18742
```

```
cat("Limite superior Economica (q33):", q33, "\n")
```

```
## Limite superior Economica (q33): 136
```

```
cat("Limite superior Intermedia (q67):", q67, "\n")
```

```
## Limite superior Intermedia (q67): 252
```

Actividad 2 — Exploración y Transformaciones para SVM

SVM utiliza distancias para determinar a qué categoría va a pertenecer un punto. Dada esta característica del modelo, es necesario escalar los datos, ya que si tenemos escalas muy distintas, ciertas variables van a tener un peso mucho mayor que otras. Por ejemplo, si las unidades de una variable están en millones y las otras están en unidades, haciendo que una domine el cálculo y que la escala termine no representando adecuadamente a las distintas propiedades del dataset. Dada esta situación, se pasa por una etapa de estandarización de los datos.

```
cat("Variables disponibles en el set de clasificacion:", ncol(train_tree_clf) - 1, "\n")
```

```
## Variables disponibles en el set de clasificacion: 29
```

```
cat("Distribucion de la variable respuesta en entrenamiento:\n")
```

```
## Distribucion de la variable respuesta en entrenamiento:
```

```
print(table(train_tree_clf$price_cat))
```

```
##  
## Economica Intermedia Cara  
##      14569      14755      14405
```

```
cat("\nDistribucion en prueba:\n")
```

```
##  
## Distribucion en prueba:
```

```
print(table(test_tree_clf$price_cat))
```

```
##  
## Economica Intermedia Cara  
##      6341      6289      6112
```

```
features_num <- train_tree_clf %>% select(-price_cat)  
resumen_escalas <- data.frame(  
  Variable = names(features_num),  
  Min = sapply(features_num, min, na.rm = TRUE),  
  Max = sapply(features_num, max, na.rm = TRUE),  
  Media = sapply(features_num, mean, na.rm = TRUE),  
  SD = sapply(features_num, sd, na.rm = TRUE)  
)  
knitr::kable(head(resumen_escalas, 10), digits = 2,  
  caption = "Resumen de escala de las primeras 10 variables")
```

Table 1: Resumen de escala de las primeras 10 variables

	Variable	Min	Max	Media	SD
host_listings_count	host_listings_count	1.00	5472.00	100.63	400.46
latitude	latitude	18.99	42.39	29.74	8.48
longitude	longitude	-159.72	-71.00	-120.44	34.02
accommodates	accommodates	1.00	16.00	4.85	2.94
bathrooms	bathrooms	0.00	17.00	1.60	0.91
bedrooms	bedrooms	0.00	48.00	1.80	1.33
beds	beds	0.00	61.00	2.67	1.99
minimum_nights	minimum_nights	1.00	365.00	7.43	17.06
maximum_nights	maximum_nights	1.00	3650.00	476.57	428.85
minimum_minimum_nights	minimum_minimum_nights	1.00	365.00	6.82	15.86

```

scale_center <- colMeans(features_num)
scale_sd <- apply(features_num, 2, sd)
scale_sd[is.na(scale_sd) | scale_sd == 0] <- 1

train_svm <- as.data.frame(scale(features_num,
                                center = scale_center,
                                scale = scale_sd))
train_svm$price_cat <- train_tree_clf$price_cat

test_features <- test_tree_clf %>% select(-price_cat)
test_svm <- as.data.frame(scale(test_features,
                                center = scale_center,
                                scale = scale_sd))
test_svm$price_cat <- test_tree_clf$price_cat

cat("Dimensiones train_svm:", nrow(train_svm), "x", ncol(train_svm), "\n")

## Dimensiones train_svm: 43729 x 30

cat("Dimensiones test_svm: ", nrow(test_svm), "x", ncol(test_svm), "\n")

## Dimensiones test_svm: 18742 x 30

```

Actividad 3 — Variable Respuesta

La variable respuesta es `price_cat`, esta representa la categoría de precio de cada una de las casas. Las categorías se establecen según los percentiles del precio 33 y 67. De esta forma, se van clasificando:

- Si es menor o igual al percentil 33, es “Económica”.
- Si está entre el percentil 33 y 67, es “Intermedia”.
- En caso contrario, es “Cara”.

Esta es la misma categorización que se ha venido utilizando en laboratorios anteriores, en donde se ha ido trabajando con modelos de categorización.

```
cat("Clases en la variable respuesta:", levels(train_svm$price_cat), "\n")
```

```
## Clases en la variable respuesta: Economica Intermedia Cara
```

```
cat("Distribucion en entrenamiento:\n")
```

```
## Distribucion en entrenamiento:
```

```
print(prop.table(table(train_svm$price_cat)))
```

```
##
```

```
## Economica Intermedia      Cara
```

```
## 0.3331656 0.3374191 0.3294153
```

Actividad 4 — Modelos SVM con Diferentes Kernels

Para no definir los hiperparámetros a prueba y error, se realiza un tuneo sobre un conjunto de hiperparámetros sobre un subconjunto de 1,500 observaciones del conjunto de entrenamiento. Esto permite identificar qué kernel y qué rango de valores de C y gamma ofrecen mejores resultados antes de entrenar los modelos completos. Se utiliza validación cruzada con cinco pliegues, lo que significa que el subconjunto se divide en cinco partes: cuatro se usan para entrenar y una para validar, rotando hasta que cada parte haya servido como validación una vez, y los resultados se promedian.

```
set.seed(123)
idx_tune <- sample(nrow(train_svm), min(1500, nrow(train_svm)))
train_tune <- train_svm[idx_tune, ]
```

```
tune_lineal <- tune(svm, price_cat ~ ., data = train_tune,
kernel = "linear",
ranges = list(cost = c(0.1, 1, 10)),
tunecontrol = tune.control(sampling = "cross", cross = 5))
```

```
tune_rbf <- tune(svm, price_cat ~ ., data = train_tune,
kernel = "radial",
ranges = list(cost = c(0.1, 1, 10),
gamma = c(0.01, 0.1, 1)),
tunecontrol = tune.control(sampling = "cross", cross = 5))
```

```
cat("Mejor configuracion lineal:\n")
```

```
## Mejor configuracion lineal:
```

```
cat("C:", tune_lineal$best.parameters$cost, "\n")
```

```
## C: 10
```

```
cat("Error CV:", round(tune_lineal$best.performance, 4), "\n\n")
```

```
## Error CV: 0.3767
```

```
cat("Mejor configuracion RBF:\n")
```

```
## Mejor configuracion RBF:
```

```
cat("C:", tune_rbf$best.parameters$cost, "\n")
```

```
## C: 10
```

```
cat("Gamma:", tune_rbf$best.parameters$gamma, "\n")
```

```
## Gamma: 0.01
```

```
cat("Error CV:", round(tune_rbf$best.performance, 4), "\n")
```

```
## Error CV: 0.3547
```

Con base en el tuneo exploratorio, el Modelo 1 usa kernel lineal con $C = 10$, y el Modelo 2 usa kernel RBF con $C = 10$ y $\gamma = 0.01$, con el objetivo de identificar si la frontera no lineal funciona mejor sobre el dataset.

```
set.seed(123)
modelo_lineal <- svm(price_cat ~ ., data = train_svm,
  kernel = "linear",
  cost = tune_lineal$best.parameters$cost,
  scale = FALSE)
```

```
cat("Modelo 1 - Kernel lineal\n")
```

```
## Modelo 1 - Kernel lineal
```

```
cat("C:", modelo_lineal$cost, "\n")
```

```
## C: 10
```

```
cat("Vectores de soporte:", nrow(modelo_lineal$SV), "\n")
```

```
## Vectores de soporte: 33028
```

```
set.seed(123)
modelo_rbf <- svm(price_cat ~ ., data = train_svm,
  kernel = "radial",
  cost = tune_rbf$best.parameters$cost,
  gamma = tune_rbf$best.parameters$gamma,
  scale = FALSE)
```

```
cat("Modelo 2 - Kernel RBF\n")
```

```
## Modelo 2 - Kernel RBF
```

```
cat("C:", modelo_rbf$cost, "\n")
```

```
## C: 10
```

```
cat("Gamma:", modelo_rbf$gamma, "\n")
```

```
## Gamma: 0.01
```

```
cat("Vectores de soporte:", nrow(modelo_rbf$SV), "\n")
```

```
## Vectores de soporte: 30168
```

Para el tercer modelo, se trató de trabajar sobre un rango de valores cercano a los hiperparámetros que habían presentado mejores resultados entre los dos modelos anteriores.

```
mejor_c <- tune_rbf$best.parameters$cost
mejor_gamma <- tune_rbf$best.parameters$gamma

set.seed(123)
tune_rbf_fino <- tune(svm, price_cat ~ ., data = train_tune,
  kernel = "radial",
  ranges = list(
    cost = c(mejor_c * 0.2, mejor_c * 0.5, mejor_c, mejor_c * 2, mejor_c * 5),
    gamma = c(mejor_gamma * 0.2, mejor_gamma * 0.5, mejor_gamma, mejor_gamma * 2)
  ),
  tunecontrol = tune.control(sampling = "cross", cross = 5))

cat("Modelo 3 - Tuneo fino RBF\n")
```

```
## Modelo 3 - Tuneo fino RBF
```

```
cat("C optimo:", tune_rbf_fino$best.parameters$cost, "\n")
```

```
## C optimo: 5
```

```
cat("Gamma optimo:", tune_rbf_fino$best.parameters$gamma, "\n")
```

```
## Gamma optimo: 0.01
```

```
cat("Error CV:", round(tune_rbf_fino$best.performance, 4), "\n")
```

```
## Error CV: 0.3533
```

```

set.seed(123)
modelo_rbf_tuned <- svm(price_cat ~ ., data = train_svm,
  kernel = "radial",
  cost = tune_rbf_fino$best.parameters$cost,
  gamma = tune_rbf_fino$best.parameters$gamma,
  scale = FALSE)

cat("Modelo 3 - Kernel RBF tuneado\n")

```

```
## Modelo 3 - Kernel RBF tuneado
```

```
cat("C:", modelo_rbf_tuned$cost, "\n")
```

```
## C: 5
```

```
cat("Gamma:", modelo_rbf_tuned$gamma, "\n")
```

```
## Gamma: 0.01
```

```
cat("Vectores de soporte:", nrow(modelo_rbf_tuned$SV), "\n")
```

```
## Vectores de soporte: 30665
```

Al comparar los dos modelos iniciales, el RBF presentó mejores resultados que la versión lineal, por lo que el tercer modelo se orientó a tunear ese kernel con una grilla más fina. Tras la optimización, los mejores parámetros encontrados fueron $C = 5$ y $\gamma = 0.01$, lo que indica que el modelo se beneficia de un margen ajustado combinado con una influencia más controlada de cada punto. Esto puede deberse, conectándolo a laboratorios anteriores, a que el dataset tiende a generar cierto sesgo al clasificar entre propiedades intermedias y económicas, con mayor dificultad para identificar correctamente las propiedades caras.

Actividad 5 — Predicción con los Modelos SVM

Con los tres modelos entrenados, se generan las predicciones sobre el conjunto de prueba para evaluar qué tan efectivo es cada uno al clasificar las propiedades en Económica, Intermedia y Cara. Se registra además el tiempo de predicción de cada modelo, dato relevante para la comparación de eficiencia en actividades posteriores.

```

tiempo_lineal <- system.time(
  pred_lineal <- predict(modelo_lineal, newdata = test_svm)
)

tiempo_rbf <- system.time(
  pred_rbf <- predict(modelo_rbf, newdata = test_svm)
)

tiempo_rbf_tuned <- system.time(

```

```

    pred_rbf_tuned <- predict(modelo_rbf_tuned, newdata = test_svm)
  )

cat("Tiempo de prediccion - Modelo 1 (Lineal): ", round(tiempo_lineal["elapsed"], 4), "s\n")

## Tiempo de prediccion - Modelo 1 (Lineal):  15.52 s

cat("Tiempo de prediccion - Modelo 2 (RBF): ", round(tiempo_rbf["elapsed"], 4), "s\n")

## Tiempo de prediccion - Modelo 2 (RBF):  38.47 s

cat("Tiempo de prediccion - Modelo 3 (RBF tuned): ", round(tiempo_rbf_tuned["elapsed"], 4), "s\n")

## Tiempo de prediccion - Modelo 3 (RBF tuned):  39.66 s

cat("Distribucion de predicciones - Modelo 1 (Lineal):\n")

## Distribucion de predicciones - Modelo 1 (Lineal):

print(table(pred_lineal))

## pred_lineal
##   Economica Intermedia      Cara
##      6988      5814      5940

cat("\nDistribucion de predicciones - Modelo 2 (RBF):\n")

##
## Distribucion de predicciones - Modelo 2 (RBF):

print(table(pred_rbf))

## pred_rbf
##   Economica Intermedia      Cara
##      6315      6777      5650

cat("\nDistribucion de predicciones - Modelo 3 (RBF tuned):\n")

##
## Distribucion de predicciones - Modelo 3 (RBF tuned):

print(table(pred_rbf_tuned))

## pred_rbf_tuned
##   Economica Intermedia      Cara
##      6321      6754      5667

```



```
cat("\nDistribucion real en prueba:\n")
```

```
##
```

```
## Distribucion real en prueba:
```

```
print(table(test_svm$price_cat))
```

```
##
```

```
## Economica Intermedia      Cara
```

```
##      6341      6289      6112
```

Al contrastar las predicciones de los tres modelos contra la distribución real del conjunto de prueba, se observa un comportamiento peculiar. Los modelos RBF logran clasificar cantidades similares de propiedades económicas e intermedias respecto a la realidad, pero se exceden en ambas categorías a costa de subestimar la categoría cara, repitiendo el patrón de sesgo identificado en laboratorios anteriores. El segundo modelo predijo 5,608 propiedades caras y el tercero 5,587, cuando el valor real es 6,119.

En cuanto al modelo lineal, presenta una dinámica distinta: sobreestima significativamente la categoría económica (6,906 frente a 6,185 reales) y subestima la intermedia (5,879 frente a 6,362 reales). Sin embargo, es el modelo que más se acerca al valor real en la categoría cara, con 5,881 predicciones frente a 6,119 reales. Las matrices de confusión de la siguiente actividad permitirán cuantificar con mayor precisión dónde se concentran los errores de cada modelo.

Actividad 6 — Matrices de Confusión

Para evaluar los modelos SVM entrenados, se generan las matrices de confusión usando el conjunto de prueba. Esto permite comparar las categorías reales contra las categorías predichas por cada modelo y observar en qué clases se presentan más errores.

```
pred_lineal <- predict(modelo_lineal, newdata = test_svm)
pred_rbf <- predict(modelo_rbf, newdata = test_svm)
pred_rbf_tuned <- predict(modelo_rbf_tuned, newdata = test_svm)

cm_lineal <- confusionMatrix(
  data = pred_lineal,
  reference = test_svm$price_cat
)

cm_rbf <- confusionMatrix(
  data = pred_rbf,
  reference = test_svm$price_cat
)

cm_rbf_tuned <- confusionMatrix(
  data = pred_rbf_tuned,
  reference = test_svm$price_cat
)

cat("Matriz de confusion - Modelo 1: SVM Lineal\n")
```

```
## Matriz de confusion - Modelo 1: SVM Lineal
```

```
print(cm_lineal$table)
```

```
##           Reference
## Prediction  Economica Intermedia Cara
## Economica   4727      1803  458
## Intermedia  1390      3046 1378
## Cara        224      1440 4276
```

```
cat("\nMetricas generales - Modelo 1: SVM Lineal\n")
```

```
##
## Metricas generales - Modelo 1: SVM Lineal
```

```
print(cm_lineal$overall)
```

```
##      Accuracy      Kappa AccuracyLower AccuracyUpper AccuracyNull
## 6.428876e-01 4.641188e-01 6.359794e-01 6.497515e-01 3.383310e-01
## AccuracyPValue McNemarPValue
## 0.000000e+00 4.365912e-29
```

```
cat("\nMetricas por clase - Modelo 1: SVM Lineal\n")
```

```
##
## Metricas por clase - Modelo 1: SVM Lineal
```

```
print(cm_lineal$byClass)
```

```
##           Sensitivity Specificity Pos Pred Value Neg Pred Value
## Class: Economica 0.7454660 0.8176760 0.6764453 0.8626850
## Class: Intermedia 0.4843377 0.7777242 0.5239078 0.7491491
## Class: Cara 0.6996073 0.8682502 0.7198653 0.8565849
##           Precision Recall F1 Prevalence Detection Rate
## Class: Economica 0.6764453 0.7454660 0.7092805 0.3383310 0.2522143
## Class: Intermedia 0.5239078 0.4843377 0.5033463 0.3355565 0.1625227
## Class: Cara 0.7198653 0.6996073 0.7095918 0.3261125 0.2281507
##           Detection Prevalence Balanced Accuracy
## Class: Economica 0.3728524 0.7815710
## Class: Intermedia 0.3102124 0.6310310
## Class: Cara 0.3169352 0.7839288
```

```
cat("\n\nMatriz de confusion - Modelo 2: SVM RBF\n")
```

```
##
##
## Matriz de confusion - Modelo 2: SVM RBF
```

```
print(cm_rbf$table)
```

```
##           Reference
## Prediction  Economica Intermedia Cara
## Economica    4713      1365  237
## Intermedia   1476      3803 1498
## Cara         152       1121 4377
```

```
cat("\nMetricas generales - Modelo 2: SVM RBF\n")
```

```
##
## Metricas generales - Modelo 2: SVM RBF
```

```
print(cm_rbf$overall)
```

```
##      Accuracy      Kappa AccuracyLower AccuracyUpper AccuracyNull
## 6.879202e-01 5.316618e-01 6.812312e-01 6.945508e-01 3.383310e-01
## AccuracyPValue McNemarPValue
## 0.000000e+00 1.236240e-16
```

```
cat("\nMetricas por clase - Modelo 2: SVM RBF\n")
```

```
##
## Metricas por clase - Modelo 2: SVM RBF
```

```
print(cm_rbf$byClass)
```

```
##      Sensitivity Specificity Pos Pred Value Neg Pred Value
## Class: Economica 0.7432582 0.8708169 0.7463183 0.8689949
## Class: Intermedia 0.6047066 0.7611820 0.5611628 0.7922273
## Class: Cara 0.7161322 0.8992082 0.7746903 0.8674763
##      Precision Recall F1 Prevalence Detection Rate
## Class: Economica 0.7463183 0.7432582 0.7447851 0.3383310 0.2514673
## Class: Intermedia 0.5611628 0.6047066 0.5821215 0.3355565 0.2029132
## Class: Cara 0.7746903 0.7161322 0.7442612 0.3261125 0.2335396
##      Detection Prevalence Balanced Accuracy
## Class: Economica 0.3369438 0.8070375
## Class: Intermedia 0.3615943 0.6829443
## Class: Cara 0.3014620 0.8076702
```

```
cat("\n\nMatriz de confusion - Modelo 3: SVM RBF Tuneado\n")
```

```
##
##
## Matriz de confusion - Modelo 3: SVM RBF Tuneado
```

```
print(cm_rbf_tuned$table)
```

```
##           Reference
## Prediction  Economica Intermedia Cara
## Economica    4684      1383  254
## Intermedia   1509      3754 1491
## Cara         148       1152 4367
```

```
cat("\nMetricas generales - Modelo 3: SVM RBF Tuneado\n")
```

```
##
## Metricas generales - Modelo 3: SVM RBF Tuneado
```

```
print(cm_rbf_tuned$overall)
```

```
##           Accuracy           Kappa AccuracyLower AccuracyUpper AccuracyNull
## 6.832248e-01 5.246210e-01 6.765098e-01 6.898830e-01 3.383310e-01
## AccuracyPValue McNemarPValue
## 0.000000e+00 1.403744e-16
```

```
cat("\nMetricas por clase - Modelo 3: SVM RBF Tuneado\n")
```

```
##
## Metricas por clase - Modelo 3: SVM RBF Tuneado
```

```
print(cm_rbf_tuned$byClass)
```

```
##           Sensitivity Specificity Pos Pred Value Neg Pred Value
## Class: Economica 0.7386848 0.8679945 0.7410220 0.8665969
## Class: Intermedia 0.5969152 0.7590942 0.5558188 0.7885385
## Class: Cara 0.7144961 0.8970705 0.7706017 0.8665392
##           Precision Recall F1 Prevalence Detection Rate
## Class: Economica 0.7410220 0.7386848 0.7398515 0.3383310 0.2499200
## Class: Intermedia 0.5558188 0.5969152 0.5756344 0.3355565 0.2002988
## Class: Cara 0.7706017 0.7144961 0.7414891 0.3261125 0.2330061
##           Detection Prevalence Balanced Accuracy
## Class: Economica 0.3372639 0.8033396
## Class: Intermedia 0.3603671 0.6780047
## Class: Cara 0.3023690 0.8057833
```

```
resumen_svm <- data.frame(
  Modelo = c("SVM Lineal", "SVM RBF", "SVM RBF Tuneado"),
  Accuracy = c(
    cm_lineal$overall["Accuracy"],
    cm_rbf$overall["Accuracy"],
    cm_rbf_tuned$overall["Accuracy"]
  ),
  Kappa = c(
    cm_lineal$overall["Kappa"],
    cm_rbf$overall["Kappa"],
    cm_rbf_tuned$overall["Kappa"]
  )
)
```

```
)

knitr::kable(
  resumen_svm,
  digits = 4,
  caption = "Comparación general de los modelos SVM"
)
```

Table 2: Comparación general de los modelos SVM

Modelo	Accuracy	Kappa
SVM Lineal	0.6429	0.4641
SVM RBF	0.6879	0.5317
SVM RBF Tuneado	0.6832	0.5246

Actividad 7 — Análisis de Sobreajuste y Desajuste

Para establecer si un modelo está sobreajustado o desajustado, comparamos la exactitud, tratando de validar el rendimiento que se da en los datos de entrenamiento contra el conjunto de prueba. Si al modelo le va muy bien clasificando datos en el de entrenamiento y luego vemos una caída considerable en el conjunto de prueba, eso quiere decir que nuestro modelo muy probablemente va a estar sobreajustado. No obstante, si el rendimiento es pésimo en ambos conjuntos, quiere decir que está desajustado.

```
pred_lineal_train <- predict(modelo_lineal, newdata = train_svm)
pred_rbf_train <- predict(modelo_rbf, newdata = train_svm)
pred_rbf_tuned_train <- predict(modelo_rbf_tuned, newdata = train_svm)
```

```
acc <- function(real, pred) mean(as.character(real) == as.character(pred))
```

```
real_train <- train_svm$price_cat
real_test <- test_svm$price_cat
```

```
tabla_ajuste <- data.frame(
  Modelo = c("Lineal", "RBF", "RBF Tuneado"),
  Acc_Train = round(c(acc(real_train, pred_lineal_train),
                        acc(real_train, pred_rbf_train),
                        acc(real_train, pred_rbf_tuned_train)), 4),
  Acc_Test = round(c(acc(real_test, pred_lineal),
                      acc(real_test, pred_rbf),
                      acc(real_test, pred_rbf_tuned)), 4)
)
```

```
tabla_ajuste$Diferencia <- round(tabla_ajuste$Acc_Train - tabla_ajuste$Acc_Test, 4)
```

```
knitr::kable(tabla_ajuste, col.names = c("Modelo", "Accuracy Train", "Accuracy Test", "Diferencia"),
  caption = "Comparación de exactitud en entrenamiento y prueba")
```

Table 3: Comparación de exactitud en entrenamiento y prueba

Modelo	Accuracy Train	Accuracy Test	Diferencia
Lineal	0.6441	0.6429	0.0012
RBF	0.7134	0.6879	0.0255
RBF Tuneado	0.7041	0.6832	0.0209

Tras analizar el rendimiento de todos los modelos generados con los datos de entrenamiento y los datos de prueba, se logra evidenciar un comportamiento en donde no hay sobreajuste. Se puede llegar a esta conclusión dado que la diferencia es bastante limitada entre la precisión en entrenamiento y la precisión en prueba. Por ejemplo, en el modelo lineal la diferencia es de tan solo 0.001, en el RBF es de 0.0162 y en el RBF tuneado de 0.0142. Al ser diferencias tan bajas, sin una brecha considerable entre ambos conjuntos, se puede afirmar que los modelos generalizan correctamente. Asimismo, dado que los resultados se mantienen por encima de 0.6 en prueba, tampoco se puede concluir que existe desajuste en ninguno de los tres modelos.

Actividad 8 — Comparación entre Modelos SVM

```

extraer_f1 <- function(cm, clase) {
  if (is.null(dim(cm$byClass)) || !"F1" %in% colnames(cm$byClass)) {
    return(NA_real_)
  }
  fila <- grep(clase, rownames(cm$byClass), fixed = TRUE)
  if (length(fila) == 0) return(NA_real_)
  as.numeric(cm$byClass[fila[1], "F1"])
}

tabla_comp <- data.frame(
  Modelo = c("Lineal", "RBF", "RBF Tuneado"),
  Accuracy = c(
    as.numeric(cm_lineal$overall["Accuracy"]),
    as.numeric(cm_rbf$overall["Accuracy"]),
    as.numeric(cm_rbf_tuned$overall["Accuracy"])
  ),
  Kappa = c(
    as.numeric(cm_lineal$overall["Kappa"]),
    as.numeric(cm_rbf$overall["Kappa"]),
    as.numeric(cm_rbf_tuned$overall["Kappa"])
  ),
  F1_Economica = c(
    extraer_f1(cm_lineal, "Economica"),
    extraer_f1(cm_rbf, "Economica"),
    extraer_f1(cm_rbf_tuned, "Economica")
  ),
  F1_Intermedia = c(
    extraer_f1(cm_lineal, "Intermedia"),
    extraer_f1(cm_rbf, "Intermedia"),
    extraer_f1(cm_rbf_tuned, "Intermedia")
  ),

```

```

F1_Cara = c(
  extraer_f1(cm_lineal, "Cara"),
  extraer_f1(cm_rbf, "Cara"),
  extraer_f1(cm_rbf_tuned, "Cara")
),
Errores = c(
  sum(as.character(pred_lineal) != as.character(test_svm$price_cat)),
  sum(as.character(pred_rbf) != as.character(test_svm$price_cat)),
  sum(as.character(pred_rbf_tuned) != as.character(test_svm$price_cat))
),
Tiempo_pred = c(
  as.numeric(tiempo_lineal["elapsed"]),
  as.numeric(tiempo_rbf["elapsed"]),
  as.numeric(tiempo_rbf_tuned["elapsed"])
)
)

knitr::kable(tabla_comp,
  col.names = c("Modelo", "Accuracy", "Kappa", "F1 Economica",
                "F1 Intermedia", "F1 Cara", "Errores", "Tiempo pred (s)"),
  caption = "Comparación de métricas entre los tres modelos SVM",
  digits = 4
)

```

Table 4: Comparación de métricas entre los tres modelos SVM

Modelo	Accuracy	Kappa	F1		F1		Errores	Tiempo pred (s)
			Economica	F1 Intermedia	Cara			
Lineal	0.6429	0.4641	0.7093	0.5033	0.7096	6693		15.52
RBF	0.6879	0.5317	0.7448	0.5821	0.7443	5849		38.47
RBF	0.6832	0.5246	0.7399	0.5756	0.7415	5937		39.66
Tuneado								

Al comparar los tres modelos, el mejor comportamiento se determina con base en el **Accuracy**, el **Kappa** y el **F1** por clase. En general, el kernel RBF permite representar fronteras más flexibles que el kernel lineal, por lo que suele mejorar cuando la separación entre las categorías de precio no es completamente recta. Aun así, el resultado más importante no es solo cuál sube unas décimas en exactitud, sino cuál mantiene un comportamiento más estable entre las tres clases.

La categoría que tiende a generar más errores es **Intermedia**, porque funciona como una zona de transición: varias propiedades comparten características con las económicas y con las caras. Por eso, aunque un modelo tenga buen rendimiento global, es importante revisar esta clase con cuidado y no quedarse únicamente con el accuracy general.

Actividad 9 — Comparación con Modelos de Clasificación Anteriores

Para comparar SVM contra los algoritmos trabajados anteriormente, se entrenan nuevamente modelos base usando el mismo conjunto de entrenamiento y prueba. Esto permite que la comparación sea más justa, porque todos los modelos se evalúan con la misma partición. En el caso de Random Forest y KNN se

usa una muestra reproducible del conjunto de entrenamiento para reducir el costo computacional, pero las métricas se calculan sobre los conjuntos completos de entrenamiento y prueba.

```
set.seed(123)

modelo_arbol_clf <- rpart(
  price_cat ~ .,
  data = train_tree_clf,
  method = "class",
  control = rpart.control(cp = 0.005, minsplit = 30)
)

n_rf_clf <- min(10000, nrow(train_tree_clf))
train_rf_clf <- train_tree_clf[sample(seq_len(nrow(train_tree_clf)), n_rf_clf), ]

modelo_rf_clf <- randomForest(
  price_cat ~ .,
  data = train_rf_clf,
  ntree = 150,
  mtry = max(1, floor(sqrt(ncol(train_rf_clf) - 1))),
  importance = FALSE
)

modelo_nb_clf <- e1071::naiveBayes(
  price_cat ~ .,
  data = train_tree_clf,
  laplace = 1
)

n_knn_clf <- min(8000, nrow(train_svm))
train_knn_clf <- train_svm[sample(seq_len(nrow(train_svm)), n_knn_clf), ]

modelo_knn_clf <- caret::knn3(
  price_cat ~ .,
  data = train_knn_clf,
  k = 7
)

modelo_logit_clf <- nnet::multinom(
  price_cat ~ .,
  data = train_svm,
  trace = FALSE,
  maxit = 300,
  MaxNWts = 10000
)

pred_arbol_train <- predict(modelo_arbol_clf, newdata = train_tree_clf, type = "class")
pred_arbol_test  <- predict(modelo_arbol_clf, newdata = test_tree_clf, type = "class")

pred_rf_train <- predict(modelo_rf_clf, newdata = train_tree_clf, type = "class")
pred_rf_test  <- predict(modelo_rf_clf, newdata = test_tree_clf, type = "class")

pred_nb_train <- predict(modelo_nb_clf, newdata = train_tree_clf, type = "class")
pred_nb_test  <- predict(modelo_nb_clf, newdata = test_tree_clf, type = "class")
```



```

pred_knn_train <- predict(modelo_knn_clf,
                          newdata = train_svm %>% select(-price_cat),
                          type = "class")
pred_knn_test  <- predict(modelo_knn_clf,
                          newdata = test_svm %>% select(-price_cat),
                          type = "class")

pred_logit_train <- predict(modelo_logit_clf, newdata = train_svm, type = "class")
pred_logit_test  <- predict(modelo_logit_clf, newdata = test_svm, type = "class")

tabla_clf <- data.frame(
  Modelo = c("Árbol de decisión", "Random Forest", "Naive Bayes",
             "KNN", "Regresión logística", "SVM RBF Tuneado"),
  Accuracy = c(
    calc_acc(train_tree_clf$price_cat, pred_arbol_train),
    calc_acc(train_tree_clf$price_cat, pred_rf_train),
    calc_acc(train_tree_clf$price_cat, pred_nb_train),
    calc_acc(train_svm$price_cat, pred_knn_train),
    calc_acc(train_svm$price_cat, pred_logit_train),
    calc_acc(train_svm$price_cat, pred_rbf_tuned_train)
  ),
  Accuracy_Test = c(
    calc_acc(test_tree_clf$price_cat, pred_arbol_test),
    calc_acc(test_tree_clf$price_cat, pred_rf_test),
    calc_acc(test_tree_clf$price_cat, pred_nb_test),
    calc_acc(test_svm$price_cat, pred_knn_test),
    calc_acc(test_svm$price_cat, pred_logit_test),
    calc_acc(test_svm$price_cat, pred_rbf_tuned)
  )
)

tabla_clf <- tabla_clf %>%
  mutate(
    Brecha = Accuracy - Accuracy_Test
  ) %>%
  arrange(desc(Accuracy_Test))

knitr::kable(
  tabla_clf,
  col.names = c("Modelo", "Accuracy Train", "Accuracy Test", "Brecha"),
  caption = "Comparación de exactitud entre modelos de clasificación",
  digits = 4
)

```

Table 5: Comparación de exactitud entre modelos de clasificación

Modelo	Accuracy Train	Accuracy Test	Brecha
Random Forest	0.7811	0.7175	0.0636
SVM RBF Tuneado	0.7041	0.6832	0.0209
Regresión logística	0.6423	0.6406	0.0017
KNN	0.6354	0.6115	0.0240
Árbol de decisión	0.6088	0.6090	-0.0002

Modelo	Accuracy Train	Accuracy Test	Brecha
Naive Bayes	0.4602	0.4651	-0.0049

Con esta comparación se puede revisar el rendimiento de SVM en relación con otros modelos clásicos de clasificación. Random Forest suele ser un competidor fuerte porque combina muchos árboles y reduce parte de la variabilidad de un solo árbol de decisión. SVM, por otro lado, aprovecha bien el escalamiento de variables y puede construir fronteras de decisión más flexibles, especialmente con kernel RBF. En este caso, el mejor modelo de clasificación se selecciona por su desempeño en prueba, ya que esa métrica representa mejor cómo se comportaría el modelo con propiedades nuevas.

Actividad 10 — Tabla Comparativa de Sobreajuste (Todos los Modelos)

Para analizar sobreajuste no basta con mirar qué modelo tiene el accuracy más alto. Lo que se debe comparar es el comportamiento en entrenamiento contra el comportamiento en prueba. Si el rendimiento en entrenamiento es bastante mayor que en prueba, el modelo probablemente aprendió detalles muy específicos del conjunto de entrenamiento y no generaliza tan bien. Si ambos rendimientos son bajos, el problema se acerca más a desajuste.

```
tabla_sobreajuste_clf <- data.frame(
  Modelo = c("Árbol de decisión", "Random Forest", "Naive Bayes", "KNN",
             "Regresión logística", "SVM Lineal", "SVM RBF", "SVM RBF Tuneado"),
  Accuracy_Train = c(
    calc_acc(train_tree_clf$price_cat, pred_arbol_train),
    calc_acc(train_tree_clf$price_cat, pred_rf_train),
    calc_acc(train_tree_clf$price_cat, pred_nb_train),
    calc_acc(train_svm$price_cat, pred_knn_train),
    calc_acc(train_svm$price_cat, pred_logit_train),
    calc_acc(train_svm$price_cat, pred_lineal_train),
    calc_acc(train_svm$price_cat, pred_rbf_train),
    calc_acc(train_svm$price_cat, pred_rbf_tuned_train)
  ),
  Accuracy_Test = c(
    calc_acc(test_tree_clf$price_cat, pred_arbol_test),
    calc_acc(test_tree_clf$price_cat, pred_rf_test),
    calc_acc(test_tree_clf$price_cat, pred_nb_test),
    calc_acc(test_svm$price_cat, pred_knn_test),
    calc_acc(test_svm$price_cat, pred_logit_test),
    calc_acc(test_svm$price_cat, pred_lineal),
    calc_acc(test_svm$price_cat, pred_rbf),
    calc_acc(test_svm$price_cat, pred_rbf_tuned)
  )
)

tabla_sobreajuste_clf <- tabla_sobreajuste_clf %>%
  mutate(
    Error_Train = 1 - Accuracy_Train,
    Error_Test = 1 - Accuracy_Test,
    Brecha_Accuracy = Accuracy_Train - Accuracy_Test,
    Diagnostico = diagnostico_clf(Accuracy_Train, Accuracy_Test)
```

```

) %>%
  arrange(desc(Accuracy_Test))

knitr::kable(
  tabla_sobreaajuste_clf,
  col.names = c("Modelo", "Accuracy Train", "Accuracy Test",
                "Error Train", "Error Test", "Brecha Accuracy", "Diagnóstico"),
  caption = "Comparación de sobreaajuste para los modelos de clasificación",
  digits = 4
)

```

Table 6: Comparación de sobreaajuste para los modelos de clasificación

Modelo	Accuracy Train	Accuracy Test	Error Train	Error Test	Brecha Accuracy	Diagnóstico
Random Forest	0.7811	0.7175	0.2189	0.2825	0.0636	Sin evidencia fuerte
SVM RBF	0.7134	0.6879	0.2866	0.3121	0.0255	Sin evidencia fuerte
SVM RBF Tuneado	0.7041	0.6832	0.2959	0.3168	0.0209	Sin evidencia fuerte
SVM Lineal	0.6441	0.6429	0.3559	0.3571	0.0012	Sin evidencia fuerte
Regresión logística	0.6423	0.6406	0.3577	0.3594	0.0017	Sin evidencia fuerte
KNN	0.6354	0.6115	0.3646	0.3885	0.0240	Sin evidencia fuerte
Árbol de decisión	0.6088	0.6090	0.3912	0.3910	-0.0002	Sin evidencia fuerte
Naive Bayes	0.4602	0.4651	0.5398	0.5349	-0.0049	Posible desajuste

La tabla permite ver de forma más clara si un modelo está aprendiendo demasiado del entrenamiento. En clasificación, los parámetros más útiles para revisar son el **Accuracy** de entrenamiento y prueba, el error en ambos conjuntos, la brecha entre ambos y, cuando se necesita más detalle, el F1 por clase. Una brecha pequeña indica que el modelo generaliza de manera razonable; una brecha grande indica posible sobreaajuste.

En este análisis, el modelo que debe elegirse no necesariamente es el que tenga el mayor rendimiento en entrenamiento, sino el que logre un buen equilibrio entre rendimiento en prueba y estabilidad. Si un modelo mejora mucho en entrenamiento, pero cae en prueba, esa mejora no es tan útil para SmartStay porque el objetivo real es estimar correctamente propiedades que todavía no se han visto.

Actividad 11 — Modelo de Regresión SVR

Para el modelo de regresión se utiliza directamente la variable **price**, porque ahora el objetivo no es clasificar las propiedades como económicas, intermedias o caras, sino estimar el precio numérico. Como SVR también depende de distancias y márgenes, las variables predictoras se estandarizan antes de entrenar el modelo. El tuneo se hace sobre una muestra del conjunto de entrenamiento para reducir el costo computacional, pero siempre conservando la misma partición general de entrenamiento y prueba.

```

train_reg <- train_num %>%
  filter(!is.na(price), is.finite(price), price >= 0)

test_reg <- test_num %>%
  filter(!is.na(price), is.finite(price), price >= 0)

x_train_reg <- train_reg %>% select(-price)
x_test_reg <- test_reg %>% select(-price)

nzv_reg <- caret::nearZeroVar(x_train_reg, names = TRUE)
if (length(nzv_reg) > 0) {
  x_train_reg <- x_train_reg %>% select(-all_of(nzv_reg))
  x_test_reg <- x_test_reg %>% select(-all_of(nzv_reg))
}

reg_center <- colMeans(x_train_reg)
reg_sd <- apply(x_train_reg, 2, sd)
reg_sd[is.na(reg_sd) | reg_sd == 0] <- 1

train_reg_svm <- as.data.frame(scale(x_train_reg, center = reg_center, scale = reg_sd))
test_reg_svm <- as.data.frame(scale(x_test_reg, center = reg_center, scale = reg_sd))

train_reg_svm$price <- train_reg$price
test_reg_svm$price <- test_reg$price

cat("Observaciones de entrenamiento para regresion:", nrow(train_reg_svm), "\n")

## Observaciones de entrenamiento para regresion: 43729

cat("Observaciones de prueba para regresion:", nrow(test_reg_svm), "\n")

## Observaciones de prueba para regresion: 18742

set.seed(456)

n_svr_tune <- min(1500, nrow(train_reg_svm))
idx_svr_tune <- sample(seq_len(nrow(train_reg_svm)), n_svr_tune)
train_svr_tune <- train_reg_svm[idx_svr_tune, ]

tune_svr <- tune(
  svm,
  price ~ .,
  data = train_svr_tune,
  kernel = "radial",
  ranges = list(
    cost = c(1, 10, 100),
    gamma = c(0.001, 0.005, 0.01),
    epsilon = c(0.1, 0.5, 1)
  ),
  tunecontrol = tune.control(sampling = "cross", cross = 5)
)

cat("Mejores parametros del SVR:\n")

```

```
## Mejores parametros del SVR:
```

```
print(tune_svr$best.parameters)
```

```
## cost gamma epsilon  
## 3 100 0.001 0.1
```

```
cat("\nError de validacion cruzada:", round(tune_svr$best.performance, 4), "\n")
```

```
##  
## Error de validacion cruzada: 1838734
```

```
set.seed(456)
```

```
n_svr_final <- min(5000, nrow(train_reg_svm))  
idx_svr_final <- sample(seq_len(nrow(train_reg_svm)), n_svr_final)  
train_svr_final <- train_reg_svm[idx_svr_final, ]
```

```
tiempo_svr_train <- system.time(  
  modelo_svr <- svm(  
    price ~ .,  
    data = train_svr_final,  
    kernel = "radial",  
    cost = tune_svr$best.parameters$cost,  
    gamma = tune_svr$best.parameters$gamma,  
    epsilon = tune_svr$best.parameters$epsilon,  
    scale = FALSE  
  )  
)
```

```
tiempo_svr_pred <- system.time({  
  pred_svr_train <- as.numeric(predict(modelo_svr, newdata = train_reg_svm))  
  pred_svr_test <- as.numeric(predict(modelo_svr, newdata = test_reg_svm))  
})
```

```
pred_svr_train <- pmax(0, pred_svr_train)  
pred_svr_test <- pmax(0, pred_svr_test)
```

```
metricas_svr <- calc_reg_row(  
  "SVR radial",  
  train_reg_svm$price,  
  pred_svr_train,  
  test_reg_svm$price,  
  pred_svr_test  
)
```

```
knitr::kable(  
  metricas_svr,  
  caption = "Métricas del modelo SVR",  
  digits = 4  
)
```

Table 7: Métricas del modelo SVR

Modelo	RMSE_Train	RMSE_Test	MAE_Train	MAE_Test	R2_Train	R2_Test	Brecha_RMSE	Brecha_R2
SVR radial	2613.793	2275.522	272.9317	238.0744	0.0014	0.0037	-338.2714	-0.0023

```
cat("Tiempo de entrenamiento SVR:", round(tiempo_svr_train["elapsed"], 4), "s\n")
```

```
## Tiempo de entrenamiento SVR: 5.23 s
```

```
cat("Tiempo de prediccion SVR:", round(tiempo_svr_pred["elapsed"], 4), "s\n")
```

```
## Tiempo de prediccion SVR: 21.14 s
```

El SVR radial permite capturar relaciones no lineales entre las características de las propiedades y su precio. A diferencia de una regresión lineal, no obliga a que el efecto de las variables sea completamente recto, lo cual tiene sentido para este problema porque el precio de un Airbnb puede depender de combinaciones de ubicación, capacidad, disponibilidad y características de la propiedad. La evaluación se hace con RMSE, MAE y R^2 para mantener la comparación en las mismas métricas que los modelos de regresión anteriores.

Actividad 12 — Comparación SVR con Modelos de Regresión Anteriores

Para comparar el SVR se construyen modelos de referencia sobre la misma variable respuesta (**price**). En la comparación se incluyen regresión lineal, árbol de regresión, Random Forest y KNN de regresión. Naive Bayes no se usa como modelo de regresión numérica porque su formulación vista en el laboratorio corresponde a clasificación; por eso no sería una comparación justa para predecir directamente el precio continuo.

```
set.seed(789)

train_reg_lm <- train_reg %>% select(price, all_of(names(x_train_reg)))
test_reg_lm  <- test_reg  %>% select(price, all_of(names(x_train_reg)))

modelo_lm <- lm(price ~ ., data = train_reg_lm)
pred_lm_train <- pmax(0, as.numeric(predict(modelo_lm, newdata = train_reg_lm)))
pred_lm_test  <- pmax(0, as.numeric(predict(modelo_lm, newdata = test_reg_lm)))

modelo_arbol_reg <- rpart(
  price ~ .,
  data = train_reg_lm,
  method = "anova",
  control = rpart.control(cp = 0.001, minsplit = 30)
)
pred_arbol_reg_train <- pmax(0, as.numeric(predict(modelo_arbol_reg, newdata = train_reg_lm)))
pred_arbol_reg_test  <- pmax(0, as.numeric(predict(modelo_arbol_reg, newdata = test_reg_lm)))

n_rf_reg <- min(10000, nrow(train_reg_lm))
train_rf_reg <- train_reg_lm[sample(seq_len(nrow(train_reg_lm)), n_rf_reg), ]
```

```

modelo_rf_reg <- randomForest(
  price ~ .,
  data = train_rf_reg,
  ntree = 150,
  mtry = max(1, floor(sqrt(ncol(train_rf_reg) - 1))),
  importance = FALSE
)
pred_rf_reg_train <- pmax(0, as.numeric(predict(modelo_rf_reg, newdata = train_reg_lm)))
pred_rf_reg_test  <- pmax(0, as.numeric(predict(modelo_rf_reg, newdata = test_reg_lm)))

n_knn_reg <- min(8000, nrow(train_reg_svm))
train_knn_reg <- train_reg_svm[sample(seq_len(nrow(train_reg_svm)), n_knn_reg), ]

modelo_knn_reg <- caret::knnreg(
  price ~ .,
  data = train_knn_reg,
  k = 7
)
pred_knn_reg_train <- pmax(0, as.numeric(predict(modelo_knn_reg, newdata = train_reg_svm)))
pred_knn_reg_test  <- pmax(0, as.numeric(predict(modelo_knn_reg, newdata = test_reg_svm)))

metricas_regresion <- bind_rows(
  calc_reg_row("Regresión lineal", train_reg_lm$price, pred_lm_train,
    test_reg_lm$price, pred_lm_test),
  calc_reg_row("Árbol de regresión", train_reg_lm$price, pred_arbol_reg_train,
    test_reg_lm$price, pred_arbol_reg_test),
  calc_reg_row("Random Forest", train_reg_lm$price, pred_rf_reg_train,
    test_reg_lm$price, pred_rf_reg_test),
  calc_reg_row("KNN regresión", train_reg_svm$price, pred_knn_reg_train,
    test_reg_svm$price, pred_knn_reg_test),
  metricas_svr
) %>%
mutate(
  Diagnostico = diagnostico_reg(RMSE_Train, RMSE_Test, R2_Train, R2_Test)
) %>%
arrange(RMSE_Test)

mejor_modelo_reg <- metricas_regresion[which.min(metricas_regresion$RMSE_Test), ]

knitr::kable(
  metricas_regresion,
  caption = "Comparación de modelos para predicción del precio",
  digits = 4
)

```

Table 8: Comparación de modelos para predicción del precio

Modelo	RMSE_Train	RMSE_Test	MAE_Train	MAE_Test	R2_Train	R2_Test	Brecha_RMSE	Brecha_MAE	Diagnostico
Random Forest	1439.168	1427.900	210.0997	222.6433	0.6973	0.6077	-	0.0896	Generalización aceptable
Árbol de regresión	1162.243	1436.204	201.1368	216.0693	0.8026	0.6031	273.9608	0.1994	Generalización aceptable

Modelo	RMSE_T	RMSE_P	MAE_T	MAE_P	R ² _T	R ² _P	Brecha_RMSE	Brecha_MAE	Diagnostico
KNN	2073.043	1985.666	271.6389	257.2804	0.3719	0.2414	-	0.1305	Generalización
regresión							87.3773		aceptable
Regresión	2453.658	2182.674	493.1827	461.1468	0.1201	0.0834	-	0.0367	Bajo ajuste
lineal							270.9833		
SVR radial	2613.793	2275.522	272.9317	238.0744	0.0014	0.0037	-	-	Bajo ajuste
							338.2714	0.0023	

El mejor modelo para estimar precios, según el menor RMSE en prueba, es **Random Forest**. Esta selección se hace con el conjunto de prueba porque lo que interesa es saber qué tan bien generaliza el modelo con propiedades que no usó para entrenar. Además del RMSE, también se revisa el MAE, ya que este muestra el error promedio en unidades de precio y es más fácil de interpretar.

En términos de sobreajuste, la columna de diagnóstico ayuda a revisar si el modelo tuvo un rendimiento demasiado alto en entrenamiento y luego perdió bastante rendimiento en prueba. Un buen modelo no solo debe tener bajo error, sino también una brecha razonable entre entrenamiento y prueba. Si un algoritmo tiene un RMSE muy bajo en entrenamiento pero mucho más alto en prueba, entonces probablemente aprendió patrones demasiado específicos del conjunto de entrenamiento.

Actividad 13 — Informe de Resultados

En este laboratorio se trabajó con modelos de máquinas vectoriales de soporte para dos objetivos distintos. Primero, se usó SVM como modelo de clasificación para predecir la categoría de precio de las propiedades (**Economica**, **Intermedia** y **Cara**). Después, se implementó un modelo SVR para estimar directamente el precio numérico de las viviendas.

En la parte de clasificación se probaron tres versiones principales de SVM: kernel lineal, kernel RBF y kernel RBF tuneado. El uso del kernel RBF fue importante porque el problema no parece separarse de manera completamente lineal. Al revisar las matrices de confusión, se observa que la clase **Intermedia** es la más difícil de clasificar, lo cual tiene sentido porque es una categoría que queda entre propiedades baratas y caras. En cambio, las propiedades de los extremos tienden a ser más fáciles de distinguir porque sus características suelen estar más marcadas.

Al comparar SVM con otros modelos de clasificación, el mejor modelo se identifica con base en el accuracy de prueba y la diferencia entre entrenamiento y prueba. En este tipo de análisis no conviene escoger solamente el modelo con mayor accuracy de entrenamiento, porque eso puede esconder sobreajuste. La comparación de sobreajuste muestra que el modelo más útil es el que mantiene buen desempeño en prueba sin aumentar demasiado la brecha con entrenamiento. Para el caso de SmartStay Advisors, esto es clave porque el modelo debe funcionar con propiedades nuevas, no solo con las que ya estaban dentro del dataset usado para entrenar.

En la parte de regresión se entrenó un modelo SVR radial para estimar el precio directamente. También se comparó contra modelos base como regresión lineal, árbol de regresión, Random Forest y KNN de regresión. De acuerdo con las métricas calculadas, el mejor modelo de regresión fue **Random Forest**, con un RMSE de prueba de **1427.9** y un MAE de prueba de **222.64**. Esta elección se justifica porque el RMSE penaliza más los errores grandes, algo relevante en precios de alojamiento, donde equivocarse mucho en una propiedad puede afectar directamente la recomendación que se le da al cliente.

En conclusión, los modelos más complejos tienden a capturar mejor las relaciones del dataset, pero también se deben vigilar para evitar sobreajuste. Para clasificación, el análisis debe apoyarse en accuracy, Kappa, F1 por clase y la brecha entre entrenamiento y prueba. Para regresión, la decisión debe basarse principalmente en RMSE, MAE, R² y la comparación entre error de entrenamiento y prueba. Con estos resultados, SmartStay puede avanzar hacia un sistema más confiable para clasificar propiedades por nivel de precio y estimar precios competitivos usando datos reales de Airbnb.